

RESEARCH + DESIGN

The App Store Layer

The package format, the distribution model, the trust ladder, the installer, and what Grok Bot and Apple are actually shipping. Everything from the second half of the session.

01

The package format — don't invent one

Two published vendor-neutral specs already exist, with a technical charter and a reference validator. Your own ledger said "BUILD YOUR EXTENSIONS, reuse plugin standards underneath." Here is the standard.

Agent Plugins 1.0.0 — agent-plugins.org

plugin.json at the package root. Strict allowed-field list:

```
$schema  name  version  description  author
homepage repository license keywords extensions
```

- Bundles exactly two component types in v1: **Skills** (from skills/) and **MCP servers** (via mcp.json)
- Directory-based, not archives. SemVer recommended.
- Path containment — anything resolving outside the plugin root is rejected
- Subprocess isolation via PLUGIN_ROOT and PLUGIN_DATA
- \$schema declares the target spec version; a schema change requires a new spec release

THE GAP Straight from the spec: **"The specification contains no explicit capability or permission system."** That's where you go, and extensions is the designed entry point.

Agent Skills — agentskills.io

SKILL.md with YAML frontmatter: name, description (max 1024 chars), license, compatibility, metadata, allowed-tools.

Progressive disclosure, which maps exactly onto your dumb-agent design:

~100 tokens	name + description, loaded at startup for EVERY skill	→ routing
<5000 tokens	the SKILL.md body, loaded only on activation	→ instructions
as needed	scripts/ references/ assets/	→ resources

That first tier *is* your router. And the description doubles as the trigger rule — the real Neon skill reads as a list of phrases: *"object storage" or "S3-compatible storage" → ...; "database" or "Postgres" → ...*

Reference validator: `skills-ref validate ./my-skill`.

Your VAPP manifest

```
{
  "$schema": "https://agent-plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "restaurant-menu",
  "version": "1.2.0",
  "description": "QR menu with verified item ratings.",
  "license": "Apache-2.0",
  "extensions": {
    "io.anextgent": {
      "vertical": "restaurant",
      "reads": ["item.list", "review.aggregate_by_item", "business.get"],
      "writes": ["visit.record_scan"],
      "surfaces": { "public_page": {...}, "qr": {...}, "agent_api": {...} },
      "settings": { "show_ratings": {...} },
      "requires": { "ingest": ["pos"] },
      "events": { "emits": [...], "listens": [...] }
    }
  }
}
```

Five things you get free: a VAPP is a valid Agent Plugin and installs in any conformant client · skilla is your installer · the local registry.json pattern is your install table · path containment and PLUGIN_ROOT are your sandbox boundary, already specified · skills-ref is a validator you didn't write.

Caveat: both specs are young. Check governance and adoption before depending on their roadmap — though extensions means your fields don't ride on their evolution either way.

02

The distribution model — steal xAI's

Read from the actual xai-org/plugin-marketplace repo, not from marketing.

The repo is an index, not a store.

From their CONTRIBUTING: *"This repo is an index: a PR doesn't ship a plugin's source."*
They host nothing. The catalog holds pointers.

```
{
  "name": "vercel",
  "description": "Vercel deployment platform integration...",
  "category": "deployment",
  "source": {
    "source": "url",
    "url": "https://github.com/vercel/vercel-plugin.git",
    "sha": "df0f55213f7b8db23a3ee7f27511ed344cdb2c74"
  },
  "homepage": "...", "keywords": [...], "domains": ["vercel.com"]
}
```

This is the direct answer to *"I want the least control possible."* You don't hold the code. You hold the index and the signature. Zero hosting cost, zero distribution liability, the developer keeps ownership — and you still control what enters the catalog.

Four mechanics to copy exactly

- **Full 40-character commit SHA.** No branches, no tags, no abbreviated SHAs — the validator rejects all of them. Their reason: *"A moving ref would let a later force-push ship new code to everyone silently."* One line of validation, enormous value.

- **Generated index.** `generate-plugin-index.py` fetches each pinned SHA and extracts the real components. The catalog metadata is *derived* from the pinned code, so it can't drift or lie about what a package contains.
- **Local plugins need a README and a valid manifest.** Remote ones vendor nothing.
- **CI validator** — `validate-catalog.py`. That's your certification tier one, free.

Review also checks source legitimacy — official org versus a throwaway account, repo exists, brand matches source. Cheap, manual, and it stops the obvious attacks.

THEIR GAP `plugin.json` has **no permission model** — no capabilities, no scopes, no risk levels. Trust is binary: `grok plugin install <name> --trust`. Fine for a developer's laptop. Not fine for a business's operational data. That's your improvement.

03

The trust ladder — Red Hat's four stores

NAMING TRAP RHEL "AppStream" is not an app store. In RHEL 8+ the distro splits into BaseOS and AppStream *repositories*. Since `ximion/appstream` (the freedesktop metadata standard) is also on your list, you have three AppStreams in play. Rename yours before it costs a week.

THING	LAYER	WHAT TO TAKE
Ecosystem Catalog	index	Index-not-store, certification badges visible in the listing, hardware certification as its own track
Red Hat Marketplace	commerce	Entitlement and metering bound to the catalog entry — Candlepin + Lago in your stack
Automation Hub	content	Your actual model. See below.
Developer Hub (Backstage)	developer portal	Software catalog, templates, docs. Later.

Why Automation Hub, not the container catalog

Ansible Collections are packaged, versioned capability content — modules, roles, plugins, playbooks bundled as one installable unit. That's structurally a VAPP, not an application. And the three-tier structure is exactly what you need:

GALAXY	free, public, community anyone with a GitHub account publishes no support claim, no guarantee
AUTOMATION HUB	curated, certified, supported subscription-gated explicit support lifecycle per collection
PRIVATE AUTOMATION HUB	the org runs their own signs and publishes their own content air-gap capable
clients check Hub first, fall back to Galaxy	

RED HAT	YOURS
Galaxy	community — anyone publishes, no guarantee
Automation Hub certified	verified / certified — you tested it, you support it
Private Automation Hub	the per-business catalog on their node

Certification isn't overhead. It's the revenue line.

The catalog is free discovery; certification and the support commitment are what people pay for. That's the Red Hat innovation and it's the one you're copying at the business level.

What Red Hat gives you: the backend — catalog, certification, signing, entitlement, lifecycle, tiering. The boring, hard-to-design half, already solved.

What it doesn't: the frontend. Their stores serve enterprise IT buyers evaluating vendor software over weeks. There is no Red Hat equivalent of a per-business grant screen. Nobody in that lineage has ever had to make install feel like nothing.

04

The installer — you already listed it

junior/skilla — item #107 on your build list. MIT, pure bash, needs only git and jq. It already does the whole job:

- Clones a repo, discovers `skills/<name>/SKILL.md`
- Resolves declared `requires:` dependencies
- Installs to `.agents/skills/` (project) or `~/.agents/skills/` (global)
- **Tracks installs in a small `registry.json`** so list, update and remove work cleanly
- For Agent Plugins 1.0.0 packages: validates the manifest, installs the whole package, expands `${PLUGIN_ROOT}` and `${PLUGIN_DATA}` in MCP config

DON'T BUILD An installer. That `registry.json` pattern is your `install` table on the node — proven, tiny, and already written.

05

Three targets per app

Apple ships one artifact: a binary that runs on a device. Yours ships up to three deployment targets in one package, and the store has to understand all of them.

NODE half	runs in WASM on the business's node ingestion parsers · capability providers · automations scheduled jobs · event handlers sees private data, never leaves the node
PUBLIC half	rendered by the edge from declared blocks page · widget · QR target · JSON-LD · MCP ZERO auth, fully cacheable, consumed by people who are not your customers and will never log in
OWNER half	wall screen tiles and phone app views also declared blocks, rendered by the platform

It maps cleanly onto what Agent Plugins already gives you:

skills/	→ what the agent can do	(node half)
mcp.json	→ tools it exposes	(node half)
extensions.io.anextgent		
└ reads / writes	→ capability wiring	(node half)
└ surfaces.public_page	→ blocks	(public half)
└ surfaces.qr / widget	→ blocks	(public half)
└ settings	→ owner UI schema	(owner half)

Your store is also **three-sided** where Apple's is two: developer → business → that business's customers. Certification checks all three halves; the grant screen covers only the node half, because the public half touches nothing private by construction.

06

Nine deployable units

Contracts make things go together. Deployments make them separate. One repo everything depends on, that depends on nothing.

1	contracts	schemas, capability defs, manifest, ActionSpec
2	app-store	Agent Plugins registry: catalog, SHA pinning, generated index, validator, CI
3	flow-engine	map once → replay → fingerprint → verify → push
4	ingest	email → structured business data, versioned parsers
5	record + resolver	canonical truth with provenance and conflict policy
6	edge	record → public page, widget, JSON-LD, MCP
7	control-plane	capability registry, policy, constitution, ledger
8	voice	Wyoming pipeline wired to a structured record
9	os	bootc image, kiosk shell, app store, recovery

Nine projects that compose — the way Foreman, Katello, Pulp and Candlepin compose. Each one is something someone else would run without the rest.

Five rules that make it actually modular

1. **No shared database between components.** If two read the same tables they're one component wearing two names. This is the rule that breaks first and costs most.
2. **Every dependency crosses a contract.** A never imports B — it calls a declared interface with a fake available for testing.
3. **Each ships its own container and starts alone.** If it crashes without another running, the boundary is wrong.
4. **Each has its own version and cadence.** contracts is semver'd; everything declares which version it implements.
5. **Each has a README that doesn't mention the others.** That's the real test.

WARNING Modular **boundaries** from day one. Modular **deployment** only when you need it. Nine services with a team of one means you spend the year on plumbing. Define the boundaries, run them in one process, split later when you've felt the pain.

The App Store is the right one to build separately first — zero dependency on the record, the node, or any app existing. It forces the manifest to be right before anything is built on it, and it's the piece with the clearest open-source story.

Grok Bot

Launched 11 August 2026, early beta. Template marketplace 28 August. Grok Build plugin marketplace June 2026, 220 plugins.

What it is

"An agent with a computer" — a **cloud-based desktop** with its own filesystem, terminal and apps. Role-based bots with their own logins, always-on. Uses apps, browses the web, writes and runs code. Taught by prompts **or screen recordings**. Bots trigger each other. Routines and event triggers. MCP servers, plugins, skills — Gmail, Calendar, Drive, multiple accounts per service.

Control: natural-language permission rules, a **separate review agent that approves or blocks**, allow/block lists, explicit authorization for significant actions, escalation to humans.

Access: requires a \$120–200/mo Cursor or SuperGrok Heavy subscription. Cursor Pro lists it at \$20/mo, Pro+ \$60, Ultra \$200, with unpublished weekly allowances. **Grok Bot is not open source** — Grok Build (the coding agent and TUI) is, and the marketplace index is.

The overlap is real

- Agent with its own computer and its own logins
- **Taught by screen recording** — that's record-and-replay
- **Natural-language permission rules** — that's a Constitution
- **A separate reviewer that approves** — actor isn't approver, same principle as verifier ≠ executor
- Human escalation, a template marketplace, routines and event triggers

Seven ways it is structurally not your product

1. **It's a cloud computer.** The thing you're positioned against, and they can't flip it — cloud is their business model.
2. **The buyer is a knowledge worker.** Personal CRM, fitness coaching, code generation, searching Slack and Notion. Not a charter captain.
3. **The plugins are enterprise SaaS.** Workspace, Slack, M365, Salesforce, Jira. Not Toast, Square, FareHarbor, Vagaro, Clover — the exact set small businesses run.
4. **No physical device, no real SIM.** A cloud desktop cannot operate an Android app needing a logged-in mobile session with SMS 2FA on the business's own number.

5. **No verification layer.** The reviewer approves *before*. Nothing independently confirms the world changed *after*.
6. **No business data plane.** No canonical truth, no verified reviews, no availability aggregation, no consented grant model.
7. **No payment rail.**

LIABILITY Their terms put account risk on the customer. A business banned from Toast for automated access on a cloud VM is the business's problem. Yours: their device, their logins, human speed, a person approving each action, and a screenshot of what happened. Same activity, very different risk profile — and that's a sales answer, not just a legal one.

08

Apple

September 9, 2026 event. Foldable iPhone (Duo/Ultra) at \$1,999, iPhone 18/18 Pro, AirPods 5, Watch Series 12 — with John Ternus debuting as CEO.

They are building a version of this, and it's gated

- **Siri rebuilt** — LLM-based, multi-step requests, running on **Google Gemini models underneath**
- **App Intents is the mechanism.** WWDC 2026 declared it "the central building block" of agentic iOS. Siri reaches into an app, retrieves information and invokes actions *without opening it*. Shipping with the fall OS releases.
- **Apple Business** — Business Connect, Business Manager and Business Essentials merged into one platform in April 2026. Siri and Maps pull recommendations directly from it, across a billion devices.
- **Maps ads launched summer 2026** in the US and Canada, sold through the same platform.

The gate, precisely

App Intents = a capability registry. Apple Business = a business record. Agentic Siri = an orchestrator. Same architecture. But both sides require **voluntary adoption on Apple's terms**: the developer must implement App Intents, and the business must claim and maintain a listing.

Apple cannot operate an app that didn't cooperate. You can. That's not a gap they can close — it's a consequence of being the platform.

58% of U.S. businesses haven't claimed their Apple listing. Only 16% actively manage it.

Apple has a billion devices, free tooling, Siri distribution and an ads business riding on it — and 84% of businesses don't maintain the data. Not because they don't care. Because maintaining it is unpaid work.

That is exactly what your text-back toggle solves. **Apple has the distribution and no mechanism. You have the mechanism.**

Three pieces of ammunition

- **Maps ads are real and shipped this summer.** Apple now monetizes the listing — renting a business access to its own presence. Fact, not rhetoric. It belongs in the 1984 remake.
- **Siri runs on Gemini.** A \$3-trillion company rented someone else's model. Models are commodity; the substrate isn't.
- **Apple just made local business data strategically important.** Every small business is about to hear they need to manage their Apple listing. You arrive with the thing that does it automatically.

MOVE **Feed Apple, don't fight it.** Apple Business becomes one more syndication destination. Your businesses rank in Siri while competitors' listings sit unclaimed — and the sales line is *"Siri is answering questions about you right now with bad data. I fix that too."*

Where they genuinely threaten you: the consumer discovery query. If Siri answers "who has crab legs nearby" from Apple Business data across a billion devices, that's your query on their distribution. It only works when the data is current, and 84% of it isn't.

The export and the bookkeeper

The box structures their private data into something they own — a spreadsheet, regenerated every morning, sitting on their machine.

- **It makes "you own it" legible.** A charter captain doesn't understand a consented federated data plane. He understands a file with all his stuff in it.
- **It's the shutdown test answered with an object.** "If I go out of business your machine keeps working" is a claim. A spreadsheet updated this morning is proof.
- **It kills API dependency for reads permanently.** No rate limits, no partner program, no access to lose.
- **Spreadsheets are where every small business already lives** — their bookkeeper, accountant, lender and insurance agent all work in Excel.
- **Portability increases retention.** Products that make leaving easy keep people.

Four design calls

1. **The spreadsheet is an export, not the source.** Canonical stays in Postgres on the node; the workbook is regenerated from it. Otherwise you inherit corruption and no referential integrity.
2. **Local file by default, not Google Sheets.** Sheets sends the data to Google, which contradicts everything else. Optional sync to a destination they pick.
3. **One workbook per domain, stable schema.** Never reorder or rename columns — their accountant has formulas pointed at those cells.
4. **The agent queries the database, not the spreadsheet.** Humans read the workbook.

Bookkeepers are an unworked channel

Each one serves 30–80 small businesses. Every month they chase numbers scattered across four systems — it's literally their billable time being wasted. Hand them a client whose numbers arrive clean and on time and they will ask what it is.

One bookkeeper who likes it is fifty warm introductions, from someone the owner already trusts with their money. Same shape as the chamber play, better qualified, and nobody is competing for that relationship.

The line: *"Every month your bookkeeper asks for numbers you have to dig out of four different systems. This just has them. Updated every morning, in a spreadsheet, on your computer."*

Research current as of September 2026. Product names, pricing and feature claims should be re-verified against first-party sources before use in legal, investment or public marketing contexts. Companions: the Playbook, the Build Spec, the Build Plan, the Parts Catalog, An Honest Read.